

INTERVIEW DEFENSE DOCUMENT

MetricLens

DataFrame-native metric movement decomposition.

Exact algebraic identity: $\text{mix_effect} + \text{rate_effect} + \text{cross_term} = \text{total_delta}$.

Metric	Value
Metric types	RatioMetric, SumMetric, CountMetric, AverageMetric
Decomposition	$\text{mix_effect} + \text{rate_effect} + \text{cross_term} = \text{total_delta}$ (exact)
Input validation checks	10 automatic per analyze() call
Output formats	JSON, Markdown, HTML
Demo dataset	756 rows, 4 dimensions (channel/device/city/category)
Tests	19 passing
PyPI	<code>pip install metriclens</code>
Dependencies	pandas, numpy only

Sidharth Kriplani | May 2026

Section 1 — Hard Interview Q&A

Q1 — The decomposition identity is $\text{total_delta} = \text{mix_effect} + \text{rate_effect} + \text{cross_term}$, and you claim it is exact. Prove it for a two-segment case.

For a ratio metric (e.g., CVR = orders/sessions), the total delta is $\text{CVR_current} - \text{CVR_baseline}$. For segment i with weight w_i (share of total) and rate r_i (CVR), the total rate is $\sum(w_i * r_i)$. The mix effect is $\sum((w_i_current - w_i_baseline) * r_i_baseline)$ — what the delta would be if only segment sizes changed. The rate effect is $\sum(w_i_baseline * (r_i_current - r_i_baseline))$ — what the delta would be if only rates changed. The cross term is $\sum((w_i_current - w_i_baseline) * (r_i_current - r_i_baseline))$ — the interaction. Summing all three: $\text{mix} + \text{rate} + \text{cross} = \sum(w_i_c * r_i_c) - \sum(w_i_b * r_i_b) = \text{CVR_current} - \text{CVR_baseline} = \text{total_delta}$. The identity holds algebraically for any number of segments, with no residual, provided weights sum to 1.0 in both periods and all segments are zero-filled for new/disappeared entries.

Q2 — Why does MetricLens zero-fill disappeared segments rather than ignoring them?

Ignoring disappeared segments breaks the exact identity. If segment S exists in the baseline period (weight=0.10, rate=0.08) but not in the current period, simply dropping it from the current-period calculation leaves the sum of current weights at 0.90 instead of 1.0. The mix effect and rate effect then no longer sum to total_delta — there is a residual equal to the contribution of the dropped segment. Zero-filling treats disappeared segments as having weight=0.0 and rate=0.0 in the current period, which correctly represents that they contributed zero volume. This produces: $\text{mix_effect for that segment} = (0.0 - 0.10) * 0.08 = -0.008$, a negative mix effect reflecting that the segment shrank to zero. The identity is preserved. The same logic applies symmetrically to new segments (zero-filled in baseline).

Q3 — AverageMetric uses a different decomposition than RatioMetric even though both are computed as numerator/denominator. Why?

The computational form is the same (both compute numerator/denominator) but the semantic semantics differ. RatioMetric represents a rate across independent events (e.g., orders/sessions) where the denominator is a distinct count of occurrences. AverageMetric represents a per-entity average where the denominator is an entity count (e.g., $\text{avg_order_value} = \text{total_revenue} / \text{order_count}$). The critical difference is in segment aggregation: for RatioMetric, the segment rate is computed as $\text{segment_numerator_sum} / \text{segment_denominator_sum}$. For AverageMetric, the segment average is computed as the arithmetic mean of per-entity values within the segment. Mixing these produces subtly wrong decompositions when segment sizes vary. Using the ratio formula on an average metric conflates entity-level variation with volume effects. MetricLens uses ratio_decomposition internally for both but the user-facing separation enforces correct semantic framing.

Q4 — The cross_term is always reported, never silently discarded. Why do many decomposition implementations discard it?

Because it is small when either the mix effect or rate effect is small, and attributing it to either component produces a number that is 'close enough' for most dashboards. In practice, $\text{cross_term} = \sum((\text{delta_w_i}) * (\text{delta_r_i}))$, which is a second-order interaction. When segment weights shift by 3% and rates shift by 2%, the cross term for each segment is $0.03 * 0.02 = 0.0006$ — tiny compared to first-order effects. Many implementations either add it to the rate effect or drop it entirely, producing totals that sum to total_delta only

approximately. MetricLens reports it explicitly because: (1) the exact identity depends on it, (2) a large cross term (which occurs when both mix and rate shift substantially) is itself informative — it means the segment that grew also improved its rate, a compounding effect worth highlighting, and (3) discarding it silently produces auditable errors that compound when the output is used in downstream calculations.

Q5 — What does MetricLens NOT detect that still causes incorrect root-cause analysis?

Five things. First, causality: a large rate_effect in paid_search does not mean the paid_search channel caused the CVR drop — it could be a seasonal effect, a creative change, or a targeting shift. MetricLens decomposes the delta; it does not identify the cause. Second, statistical significance: no check determines whether the mix_effect or rate_effect is larger than sampling noise. Two periods with only 100 sessions each can produce decompositions that look precise but are entirely noise. Third, confounding dimensions: the tool decomposes one dimension at a time. If channel and device are correlated (mobile users come mostly from paid_search), the channel decomposition and device decomposition overlap. Fourth, outlier-driven decompositions: a single extreme day in a 7-day window can dominate the mix_effect for a segment. Fifth, label correctness: if the numerator column is miscounted (double-counted orders, for example), the decomposition will be mathematically correct but semantically wrong.

Q6 — The interpretation_note is mandatory and cannot be suppressed. Why is that a design choice rather than an option?

Because the most dangerous use of MetricLens output is treating a decomposition as a causal explanation and acting on it without further investigation. The note reads: 'MetricLens produces decomposition signals for investigation. It does not establish causality, statistical significance, or attribution. Treat outputs as investigation priorities, not decisions.' Making it optional creates a path for teams to suppress it in internal dashboards, where the absence of the note is invisible to stakeholders reading the report. A mandatory note cannot be accidentally omitted. This is the same design principle used in DocIngestQA's interpretation note and in clinical diagnostic tools that always display uncertainty ranges — the cost of misinterpretation exceeds the cost of adding a few lines to every output.

Q7 — A segment shows mix_effect = +0.005 and rate_effect = -0.008. How do you interpret this for a CVR metric?

The segment grew in volume (positive mix_effect) — it represents a larger share of total sessions in the current period. But its own conversion rate declined (negative rate_effect) — the sessions coming from this segment converted at a lower rate. The net total_effect = $0.005 + (-0.008) + \text{cross_term}$ is likely negative, meaning this segment is a drag on overall CVR even though it is growing. The business implication: this segment is scaling up but at lower quality. The investigation priority is the rate_effect mechanism — why did this segment's CVR fall? Candidate explanations include a change in targeting (bringing in less-intent users), a UI change that specifically hurt this segment, or a product availability issue. The mix_effect growing while rate_effect falls is a common pattern in acquisition-heavy periods where new user cohorts have lower initial conversion rates than retained users.

Q8 — What is FLAT_EPSILON and why does MetricLens use it?

FLAT_EPSILON is a small value ($1e-10$) used to detect effectively-flat metrics — cases where total_delta is so small relative to the baseline value that the decomposition output is dominated by floating-point noise. Without FLAT_EPSILON, a metric that changes from 0.07173 to 0.07174 ($\text{delta} = 0.00001$) would produce a

decomposition with apparent precision down to six decimal places, implying certainty that is not present given the sample sizes involved. When $\text{abs}(\text{total_delta}) < \text{FLAT_EPSILON} * \max(\text{abs}(\text{baseline_value}), \text{abs}(\text{current_value}))$, MetricLens flags the result as effectively unchanged and notes that the decomposition values are within floating-point precision. This prevents false investigation priorities from numerical noise masquerading as a real signal.

Q9 — How does MetricLens handle a dimension with 200 unique segment values?

It runs the decomposition across all 200 segments and returns all of them in the output, sorted by absolute `total_effect` descending. The top segment (largest driver) appears first, forming an automatic investigation priority ranking. There is no segment count limit. In practice, 200 segments with sparse data can produce noisy decompositions — segments with only 5 sessions in the baseline period will have high variance rate effects. The validation check `date_coverage_baseline` flags sparsity at the period level but not at the individual segment level. The practical recommendation is to aggregate low-frequency segments into an 'Other' bucket before passing to MetricLens, or to filter on minimum segment size using the caller's DataFrame logic before running `analyze()`. A future version may add a `minimum_segment_size` parameter to `AuditConfig`.

Q10 — A SumMetric decomposition shows `mix_effect = 0.0` for all segments. Under what condition does this happen?

When segment volume shares are identical between the baseline and current periods — that is, every segment grew or shrank proportionally. For SumMetric (e.g., total revenue), `mix_effect` for segment *i* is $(w_{i_current} - w_{i_baseline}) * \text{total_baseline}$. If $w_{i_current} = w_{i_baseline}$ for all *i* (same proportional distribution), all `mix_effects` are zero and the entire delta is explained by `rate_effects` (each segment's absolute contribution per unit share). This is also the expected output for a perfectly proportional growth scenario (all segments grow at the same rate) — in that case `rate_effects` will be uniformly positive. A real case where this occurs: a promotions campaign that lifts all channels proportionally, leaving mix unchanged. Seeing `mix_effect = 0.0` across all segments is a valid, interpretable result, not a bug.

Section 2 — What MetricLens Does Not Do

No causality. Decomposition is algebraic attribution, not causal inference. `mix_effect` and `rate_effect` show what happened, not why.

No significance testing. No p-values, confidence intervals, or sample size checks. Use a stats testing library alongside MetricLens for significance.

No multi-dimensional interaction. Each dimension is decomposed independently. Cross-dimension interactions (e.g., `mobile` x `paid_search`) require custom aggregation before calling `analyze()`.

No outlier detection. Extreme values in a single day of a 7-day window can dominate the decomposition. Outlier handling is the caller's responsibility.

No forecasting. MetricLens explains past movement. Forward-looking prediction requires a separate forecasting model.